ChatGPT

Mathematical Foundations You'll Actually Use

Graduate-level work in Computer Science, especially in machine learning and AI, demands a solid grasp of certain mathematical foundations. This study guide covers key topics that you will **actually use** in practice. We'll explore fundamental concepts like maximum likelihood and cross-entropy, perplexity and bits-per-token, KL divergence and regularization, gradient descent and backpropagation, the math of scaled dot-product attention, weight initialization and normalization, model calibration (ECE), and statistical evaluation methods (paired bootstrap and multiple comparisons). Each section provides explanations, derivations, and examples. **Exercises and reflection questions** are included at the end to reinforce understanding (feel free to leverage AI tools or coding for simulation as needed).

Introduction

Modern machine learning relies on a **probabilistic framework** and **optimization methods** under the hood. We train models by maximizing likelihoods or equivalently minimizing loss functions like cross-entropy. We evaluate models using metrics like perplexity (especially in language models) which relate to entropy in information theory. We regularize models to prevent overfitting, sometimes using concepts like KL divergence to measure divergence from desired distributions. Training deep networks demands understanding **gradient descent** for optimization and **backpropagation** for efficient gradient calculation. Architectures like Transformers use **scaled dot-product attention**, whose math we should comprehend. Proper **weight initialization** and **normalization** techniques help address issues of vanishing or exploding variance, ensuring stable training. After training, we care about **calibration** – whether a model's predicted probabilities reflect true probabilities – measured by metrics like **Expected Calibration Error (ECE)**. Finally, when comparing models or making claims, we apply **statistical tests** (e.g. paired bootstrap) and account for **multiple comparisons** to ensure our conclusions are sound. Let's dive into each of these topics in detail.

Maximum Likelihood and Cross-Entropy

Maximum Likelihood Estimation (MLE) is a fundamental principle for fitting models to data. Given data and a parametric model, MLE chooses parameters that make the observed data *most probable*. Formally, for data points $x^{(1)},\dots,x^{(N)}$ and model probability $q_\theta(x)$, MLE finds $\theta^* = \arg\max_\theta \prod_{i=1}^N q_\theta(x^{(i)})$, or equivalently $\arg\min_\theta -\sum_{i=1}^N \log q_\theta(x^{(i)})$ [1]. In supervised learning with pairs $(x^{(i)}, y^{(i)})$, we maximize the conditional likelihood $q_\theta(y^{(i)}|x^{(i)})$ [2], which again leads to minimizing the negative log-likelihood.

Cross-Entropy as a Loss: In practice, the *negative log-likelihood* is exactly the **cross-entropy loss** between the empirical data distribution and the model's predicted distribution [3] [4]. For classification, if the true distribution $p(y|x)$ is a one-hot vector (i.e. all probability on the correct class $y^{(i)}$), the cross-entropy for a single example is:

$$H(p, q_\theta) \;=\; -\sum_{y} p(y|x) \log q_\theta(y|x) \;=\; -\log q_\theta(y^{(i)}|x^{(i)}),$$

since $p(y|x^{(i)})=1$ for $y=y^{(i)}$ and 0 otherwise [5]. Summing over all examples, the total loss $L(\theta) = -\sum_{i=1}^N \log q_\theta(y^{(i)}|x^{(i)})$ [6]. Minimizing this is exactly the same as maximizing the likelihood of the data [7]. In other words, training a classifier with cross-entropy loss **is performing maximum likelihood estimation** [8]. This is why cross-entropy is the de facto training loss for classification models – it directly aligns with the MLE principle.

Relation between Cross-Entropy and KL Divergence: Cross-entropy comes from information theory and measures the "distance" between two distributions. If p is the true data distribution and q is the model distribution, the cross-entropy is $H(p,q) = \mathbb{E}_p[-\log q(x)]$ [9]. This can be decomposed as:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \parallel q),$$

where $H(p)$ is the entropy of the true distribution (a constant with respect to θ) and $D_{\mathrm{KL}}(p \parallel q)$ is the Kullback–Leibler divergence [10]. Since $H(p)$ doesn't depend on the model, **minimizing cross-entropy $H(p,q)$ is equivalent to minimizing $D_{\mathrm{KL}}(p \parallel q)$** [11] [12]. Intuitively, maximizing likelihood is "bringing the model distribution q closer to the empirical data distribution p." In fact, Goodfellow et al. (2016) describe MLE as *"minimizing the dissimilarity between the empirical distribution defined by the training set and the model distribution, with the degree of dissimilarity measured by the KL divergence… Minimizing this KL divergence corresponds exactly to minimizing the cross-entropy between the distributions."* [11]. Thus, MLE ⇔ minimize cross-entropy ⇔ minimize KL divergence to the data.

Cross-Entropy in Practice: Cross-entropy is typically measured in "nats" (if using natural $\log$) or in "bits" (if using $\log_2$). In the context of neural networks, you often see *binary cross-entropy* for two-class problems or *categorical cross-entropy* (a.k.a. softmax loss) for multi-class problems. These are specific instances of the general formula above. Cross-entropy yields a loss in bits if log base 2 is used, reflecting the average number of bits needed to encode the true labels given the model's predicted distribution [13]. For example, a perfect model would have a cross-entropy of 0 bits (it "takes 0 bits" to convey the surprise of the correct answer), whereas a very uncertain or wrong model has higher cross-entropy.

Key point: Training by cross-entropy ensures our model's predicted probabilities match the training data distribution as closely as possible. In essence, *the cross-entropy cost function* is *the negative log-likelihood.* It is ubiquitous in deep learning for classification tasks [14] [15].

Perplexity and Bits-Per-Token

When evaluating language models or any probabilistic model of sequences, **perplexity** is a common metric. Perplexity is closely related to cross-entropy and can be thought of as a measure of "surprise" or uncertainty. Formally, for a probability distribution or model q over sequences, **perplexity (PPL)** is defined as the exponentiated average negative log-likelihood of the sequence. For example, given a sequence of N tokens with model probability $q(s_1, \dots, s_N)$, the perplexity is:

$$\mathrm{PPL} = 2^{-\frac{1}{N}\log_2 q(s_1 \ldots s_N)}.$$

If we express this per token, **perplexity per token** can be written as:

$$\mathrm{PPL} = \left(\prod_{i=1}^{N} q(s_i) \right)^{-1/N},$$

which is equivalently $2^{H(p; q)}$ where $H(p; q)$ is the cross-entropy (in bits) of the true sequence distribution p relative to model q [16]. In other words, *perplexity is the exponentiated cross-entropy*. A lower cross-entropy means lower perplexity, indicating the model is less "perplexed" by the data (i.e. it assigns higher likelihood to the test set).

Interpretation: Perplexity can be understood in an intuitive way. If a model has perplexity P, it is "as confused as if it had to choose uniformly among P possibilities for each token." For example, if a language model has a perplexity of 50, it's about as uncertain on average as a fair 50-sided die – each word it predicts is like picking from 50 equally likely options. A perfectly certain model (that assigns probability 1 to the correct next token) would have perplexity 1. A higher perplexity means more uncertainty. In the context of language: the perplexity per word for an *English* model might be, say, 20 – meaning on average the model is choosing among 20 plausible words at each step.

To make this concrete, consider a classic example from the Brown Corpus: A trigram model achieved about **247 per word perplexity**, corresponding to a cross-entropy of ~7.95 bits per word [17] [18]. That means on average, each word's surprise is like picking from 247 possibilities (7.95 bits of information). If a model's cross-entropy is 7.95 bits, its perplexity is $2^{7.95} \approx 247$. Indeed, **bits-per-token** is often used interchangeably with cross-entropy; in this example 7.95 bits/word is the loss, and perplexity is 247.

Bits-Per-Character or Bits-Per-Word: You might encounter **bits-per-character (BPC)** or **bits-per-word (BPW)**, especially when comparing models at different levels (character-level vs word-level). These metrics literally measure the average number of bits required to encode each character/word according to the model [19] [20]. They are just cross-entropy in base 2. One reason to report BPC or BPW in research is to have a base-2 measure (since many frameworks compute loss in nats using natural log, converting to bits yields a more interpretable number – how many bits of uncertainty per token) [21]. As The *Gradient* explains, if everyone reports in bits, it's easier to compare results across models [22].

For example, an English character-level model might have an entropy of ~1.3 bits/char (meaning it can compress text to ~1.3 bits per character on average, reflecting English's character-level redundancy). Word-level entropy is higher (because words carry more info than a single char).

Summary: Perplexity $= 2^{\text{cross-entropy (bits)}}$ is a convenient scalar metric for model performance. It is **lower-bounded by the entropy of the true distribution** (no model can do better than the true data's entropy) [23]. A language with entropy H bits per word means no model can get perplexity below 2^H. As models improve, their perplexity approaches this theoretical limit. Always remember: **lower perplexity = better model.** When reporting perplexity or entropy, specify if it's per token and what the unit is (bits or nats), as well as any normalization (e.g. context length) [24] [25].

Aside: Perplexity is an **intrinsic evaluation** for generative models (especially language models). While useful, comparing perplexities across different tokenizations or tasks can be tricky [26]. But as a single model improves (say you reduce perplexity from 50 to 25 on the same dataset), that's a clear sign of progress.

KL Divergence and Regularization

We've already seen **Kullback–Leibler divergence (KL divergence)** appear in the context of MLE. Let's define it clearly. For two distributions $p(x)$ (true) and $q(x)$ (approximation), the KL divergence is:

$$D_{\mathrm{KL}}(p \parallel q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_{p}[\log p(x) - \log q(x)].$$

This is a measure (in nats or bits) of how one distribution diverges from another. It's *asymmetric*: $D_{\mathrm{KL}}(p \parallel q) \neq D_{\mathrm{KL}}(q \parallel p)$ in general [27]. A KL of 0 means the distributions are identical (almost everywhere). KL is always ≥ 0 by Gibbs' inequality.

In machine learning, KL divergence often shows up in both **objective functions** and **regularization terms**:

- **In objectives:** as we discussed, maximizing likelihood leads to minimizing $D_{\mathrm{KL}}(p_{\text{data}} \parallel q_{\text{model}})$. In **variational inference**, we often minimize $D_{\mathrm{KL}}(q \parallel p)$ where q is an approximate distribution to a target p (e.g. in VAEs, a *KL term* ensures the learned latent distribution doesn't stray too far from a prior). In **knowledge distillation**, one might minimize $D_{\mathrm{KL}}(q_{\text{teacher}} \parallel q_{\text{student}})$ to make a student model mimic a teacher's output distribution.

- **As regularization:** KL can be used to impose a penalty that nudges the model towards some reference distribution. A classic example is **label smoothing**, where instead of a one-hot target, you use a mixture of the one-hot and a uniform distribution. This is effectively adding a small KL divergence term that penalizes the model for deviating too far from uniform confidence, thus preventing over-confident predictions. Another example: in Bayesian approaches, adding a prior on model weights leads to an additional term in the loss which can often be interpreted as a KL divergence between the weight distribution and the prior. For instance, **L2 weight decay** corresponds to a Gaussian prior on weights; minimizing the L2 norm is equivalent to *maximizing the log-probability of weights under a zero-mean Gaussian prior* [28]. In fact, weight decay can be seen as minimizing $D_{\mathrm{KL}}(q(w)\parallel p(w))$ where $p(w)$ is a zero-mean Gaussian and $q(w)$ is a delta distribution at the learned weight value. By penalizing large weights, we keep the learned weights in a plausible range (per the prior).

To illustrate the second point: if our prior on a weight w is $N(0,\sigma^2)$, then $-\log p(w) \propto \frac{w^2}{2\sigma^2}$. Adding this to the loss is $L + \frac{\lambda}{2}w^2$, which is L2 regularization. It's no coincidence – it's the negative log-likelihood of the prior. Hinton famously noted: *"Minimizing the squared weights is equivalent to maximizing the log probability of the weights under a zero-mean Gaussian prior."* [28].

Regularization via KL divergence is powerful because it gives a principled way to avoid overfitting: you are literally telling the model to stay close to some reference behavior. For example, in a VAE the loss is:

$$\mathcal{L}_{\mathrm{VAE}} = \mathbb{E}_{q(z|x)}[-\log p(x|z)] + D_{\mathrm{KL}}(q(z|x) \parallel p(z)),$$

where the second term is a regularizer that keeps the approximate posterior $q(z|x)$ close to a prior $p(z)$ (often $N(0,I)$). This **KL regularization** term helps ensure the latent space doesn't overfit every data point individually (which would defeat the purpose of learning a smooth representation) [29].

In summary, **KL divergence** is both a theoretical measure of distribution difference and a practical tool in learning algorithms:

- It connects to cross-entropy (as $H(p,q) = H(p) + D_{KL}(p|q)$) [10], reinforcing that training reduces KL.
- It appears as a term in many modern loss functions to encode prior knowledge or preferences (Bayesian NN priors, VAEs, label smoothing, etc.).
- It's also used to analyze models: e.g. in comparing probability distributions of outputs or in measuring information gain.

One should note that *KL divergence is not a symmetric distance*, and minimizing $D_{\mathrm{KL}}(p|q)$ vs $D_{\mathrm{KL}}(q|p)$ can lead to different results – the former tends to cover all modes of p (zero penalty for covering extra areas q has that p doesn't), whereas the latter tends to focus on modes of q and ignore areas p has but q doesn't (since if $q(x)=0$ but $p(x)>0$, $D_{\mathrm{KL}}(q|p)$ is infinite, but $D_{\mathrm{KL}}(p|q)$ is finite and penalizes those). This asymmetry explains choices in variational approximation vs ML, etc.

Gradient Descent and Backpropagation

Gradient Descent is the bread-and-butter optimization algorithm for training neural networks (and many other models). The concept is simple: if you have a loss function $J(\theta)$, the gradient $\nabla_\theta J$ tells you the direction of steepest *increase*. To *minimize* the loss, you step in the opposite direction of the gradient. A basic gradient descent update is:

$$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta),$$

where η is the learning rate (step size). This update moves θ slightly in the direction that *most decreases* J. Repeating this many times is gradient descent. Stochastic gradient descent (SGD) simply means we use an estimate of the gradient from a mini-batch of data rather than the full dataset at each step.

Why gradients? The "gradient" in gradient descent refers to the derivative of the loss with respect to parameters. Computing this efficiently for complex models is critical. A neural network might have millions of weights; computing partial derivatives naively can be extremely slow. Fortunately, we have **backpropagation** – an algorithm that leverages the chain rule of calculus to compute all partial derivatives in time proportional to a forward pass.

Backpropagation (backward propagation of errors) is essentially **reverse-mode automatic differentiation** applied to the computation graph of the network [30]. It works by iteratively applying the chain rule *from the outputs back to the inputs*. Consider a simple example: a network layer or computation $z = f(x)$ followed by output $y = g(z)$. We want $\frac{\partial y}{\partial x}$. By the chain rule:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

Backprop starts at the end (output) with $\frac{\partial \text{Loss}}{\partial y}$, then multiplies by $\frac{\partial y}{\partial z}$ to get $\frac{\partial \text{Loss}}{\partial z}$, and continues backward. For an entire network, we accumulate gradients at each node using the chain rule.

A useful perspective is to imagine the neural network's computations as a **computational graph** [31]. Nodes represent intermediate values, edges represent functions/operations. Backprop sends "error signals" backward through this graph. Each node receives the gradient of the loss with respect to its output, and it multiplies by its local derivative (and sums from multiple outputs via the sum rule) to send gradients to its inputs.

Example (Chain Rule in Action): Suppose we have a simple function $f(x, y, z) = (x + y) \cdot z$. Let's manually apply backprop: - Forward pass: compute $q = x+y$; then output $f = q \cdot z$ [32]. - Backward: We want $\partial f/\partial x, \partial f/\partial y, \partial f/\partial z$. From the chain rule: - $\frac{\partial f}{\partial q} = z$ (treating $q\cdot z$, derivative wrt q) [33], - $\frac{\partial f}{\partial z} = q$ [33]. - And $\frac{\partial q}{\partial x} = 1$, $\frac{\partial q}{\partial y} = 1$ (since $q=x+y$) [33]. - Now, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = z \cdot 1 = z$. - Similarly $\frac{\partial f}{\partial y} = z$. - We already have $\frac{\partial f}{\partial z} = q = x+y$.

So the gradients are: $d_x = z, \; d_y = z, \; d_z = x+y$. If you plug in some numbers (say $x=-2, y=5, z=-4$ as in an example) [34], you can verify these. This matches the manual approach in the CS231n notes, which did exactly this and got $\frac{\partial f}{\partial x} = -4$ (since $z=-4$), $\frac{\partial f}{\partial y} = -4$, $\frac{\partial f}{\partial z} = 3$ (since $x+y=3$) [35]. Backprop essentially did:

```
df/dz = q = 3
df/dq = z = -4
dq/dx = 1, dq/dy = 1
df/dx = df/dq * dq/dx = -4 * 1 = -4
df/dy = df/dq * dq/dy = -4 * 1 = -4
```

which matches.

Computational efficiency: The brilliance of backprop is that it reuses intermediate computations and handles the exponentially many paths in the computation graph in a linear pass [36] [37]. Instead of naively summing over all possible paths (which grows combinatorially with network depth and connections), backprop aggregates gradients at each node. Both forward and backward passes typically have the same order of complexity. In practice, modern autodiff frameworks (TensorFlow, PyTorch, JAX, etc.) implement backprop for you, so you don't code it from scratch, but understanding it is crucial for debugging and developing new architectures.

Chain Rule Visualization: The figure below shows a simple computational graph and illustrates derivatives on each edge and how backprop flows. Notice how each node's gradient is the product of gradients along the paths behind it, summed over multiple paths if they exist (as per the multivariate chain rule).

Example computational graph with values and partial derivatives on each edge, illustrating how backpropagation applies the chain rule. In this simple graph, $c = a + b$ and $e = c \cdot d$ with $d = b + 1$. The edges are labeled with local derivatives. The derivative of final output e with respect to input a can be obtained by multiplying along the path $a \to c \to e$ giving 1×2 (since $\partial c/\partial a = 1$ and $\partial e/\partial c = 2$ for the given values), whereas for b there are two paths (through c and through d), which are summed: $\frac{\partial e}{\partial b} = 1 \cdot 2 + 1 \cdot 3$ in this example [38] [39].

In the figure, the two paths from b to e correspond to the two terms in $\partial e/\partial b = 1*2 + 1*3$ [39]. Backprop efficiently computes these without enumerating both paths explicitly – it *factors* the computation by accumulating gradients at node c and d first [36] [37].

In summary, gradient descent is how we *optimize*, and backpropagation is how we *compute gradients*. Together, they allow training of deep networks with millions of parameters. It's worth noting that many improvements build on basic SGD: momentum, Adam, etc., but they still use gradients at their core. Having a clear mental model of gradient flow (how errors at the output propagate back to influence each weight) is invaluable for diagnosing problems like vanishing gradients or for designing architectures (e.g., understanding how skip connections alleviate gradient vanishing by providing shorter paths).

Linear Attention Math: Scaled Dot-Product

Modern attention-based models (like Transformers) rely on the **scaled dot-product attention** mechanism. It's called "linear" in the sense of linear algebra operations (dot products and matrix multiplies), not in the sense of a linear function overall (there's a nonlinearity from softmax). Let's break down the math of scaled dot-product attention:

We have three sets of vectors in attention: **Queries (Q)**, **Keys (K)**, and **Values (V)**. Think of a Query as representing the vector for an element that wants to attend to others, Keys as representing each potential other element, and Values as the content attached to each key. In self-attention (like in a Transformer encoder), these often come from the same source, but conceptually they can be different.

For a single query q and a set of keys ${k_j}$ with corresponding values ${v_j}$, the **dot-product attention** computes a score for each key, usually via a dot product $q \cdot k_j$. Then it normalizes these scores with a softmax to get attention weights, and finally computes a weighted sum of the values. The *scaled* dot-product attention adds one tweak: divide the dot product by $\sqrt{d_k}$ (the square root of the key dimensionality). So the formula for **Scaled Dot-Product Attention** is:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)\, V.$$

In words: - Compute similarity scores $S = Q K^\top$, which produces a matrix of dot-products between each query and each key. - Scale those scores by $\frac{1}{\sqrt{d_k}}$. - Apply softmax to each row of that scaled score matrix (each query's scores across all keys) to get attention weight distribution. - Multiply by V to get the weighted sum of values for each query [40] [41].

The result has the same shape as Q (each query is transformed into a new "attended" representation).

Why the $1/\sqrt{d_k}$ scaling? It was introduced in the original Transformer paper *"Attention Is All You Need"* to counteract the effect of large dot-product values when the dimension d_k is large. Without scaling, the dot products $q \cdot k$ grow in magnitude with d_k, and thus the softmax could push into a very peaky regime (where one value dominates the rest) or cause gradients to vanish for small differences [41]. By scaling down by $\sqrt{d_k}$, we ensure that the variance of $q \cdot k$ remains roughly 1, keeping the softmax in a nice range where gradients flow well [42] [43]. In effect, **the scaling factor prevents extremely large values inside the softmax, improving training stability** [43]. (As a side note, if queries and keys have variance 1, the dot product has variance d_k, so dividing by $\sqrt{d_k}$ makes the variance ~1 again [42].)

Linear algebra view: If Q is an $n \times d_k$ matrix (n queries of dimension d_k), K is $m \times d_k$ (m keys), and V is $m \times d_v$ (m value vectors of dimension d_v), then $Q K^\top$ yields an $n \times m$ matrix of raw attention scores. Softmax applied row-wise yields an $n \times m$ matrix of attention weights (each row sums to 1). Multiplying by V (which is $m \times d_v$) yields an $n \times d_v$ result – each query is transformed into a d_v-dim output. In self-attention within a Transformer, typically $n=m$ (same set attending to itself). In cross-attention (like decoder attending to encoder outputs), n could differ from m. The operations are all matrix multiplications, which are highly optimized in deep learning libraries.

Putting it together: One can summarize scaled dot-product attention as: 1. **Similarity**: compute score $s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$ for each query i and key j. 2. **Softmax**: $a_{ij} = \frac{\exp(s_{ij})}{\sum_{j'} \exp(s_{ij'})}$, giving attention weights a_{ij}. 3. **Weighted sum**: Output for query i is $o_i = \sum_j a_{ij} v_j$.

This is done in parallel for all queries via the matrix formula above [44] [45].

Multi-head attention: In practice, Transformers use multiple parallel attention heads. Each head has its own Q, K, V linear projections and computes attention, then the results are concatenated. This gives the model the ability to attend in different "subspaces" and capture different types of relationships. The math for each head is the same as above, just performed h times (for h heads) with smaller d_k, d_v per head, and then combined.

Linear Attention (Efficient variants): The question says *"linear attention math (scaled dot product)"*, which likely refers to the standard scaled dot-product attention formula we just discussed. Note, however, that there is research on *linear-time attention approximations* (often called linear attention) to avoid the quadratic cost in n (sequence length). These methods (e.g. Performer, Linformer, etc.) approximate or reformulate the softmax attention to scale more linearly [46] [47]. Those are advanced topics, but at a high level, they try to either factorize the attention computation or use kernel approximations so that you don't explicitly compute the $n \times m$ matrix for large n. The core scaled dot-product idea remains, but with tweaks (like using a different kernel than softmax which is more factorable). As of this guide, the standard attention is quadratic, and "linear attention" methods are proposed to handle very long sequences by sacrificing exact softmax computation for an approximation.

For our purposes, **knowing the standard scaled dot-product formula is essential**, as well as why the scaling is there. Also, remember that attention is *differentiable* – it's part of the computation graph, so during backprop, gradients flow through the softmax into Q,K,V. This means a Transformer can learn to put more weight on certain keys as needed by adjusting Q,K vectors.

Finally, one more intuition: Scaled dot-product attention is essentially computing a **weighted average of values** where weights come from how well keys match the query. It can be seen as *soft content-based addressing*. If a query vector is very similar to one of the key vectors, that key's value will get a high weight. If it's similar to multiple, it'll attend to all of them proportionally. The softmax ensures weights are positive and sum to 1 (so it's like a probability distribution over keys). This mechanism allows each position in a sequence to gather information from other positions as needed, which is why attention is so powerful in sequence modeling.

Variance, Initialization, and Normalization

Training deep networks is tricky partly because of issues like **vanishing or exploding activations and gradients**. Two important techniques to address these issues are **proper weight initialization** and **normalization layers**. Both aim to control the distribution (especially the variance) of signals as they propagate through the network.

Weight Initialization and Variance

If we start a neural network with all weights = 0, every unit in a layer will produce the same output (symmetry problem) [48]. So we use **random initialization** – but not just any random: it must be scaled correctly. The goal is often to preserve the variance of activations (and gradients) as the signal flows forward (and backward) through layers [49].

Xavier/Glorot Initialization (2010): Glorot & Bengio derived an initialization scheme for neurons with linear or tanh activations that balances the variance between layers [49]. The idea is to choose the initial weights' variance as: $\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ [50]. Here n_{in} is the number of input connections to a neuron (fan-in) and n_{out} is the number of output connections (fan-out). If using a uniform distribution, this corresponds to sampling $w \sim \mathcal{U}(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}})$ [51]. If using a normal distribution, it would be $N(0, \frac{2}{n_{in}+n_{out}})$. Glorot initialization was designed to make the **variance of activations roughly equal in each layer**, avoiding growth or decay of signals [49]. When fan-in = fan-out (e.g., many hidden layers), this simplifies to $\frac{1}{n_{\text{in}}}$ which is known as LeCun's initialization for linear networks [52] [50].

He Initialization (Kaiming Initialization, 2015): For ReLU networks, it was found that the Glorot formula wasn't ideal because ReLUs are not linear (they zero-out half the inputs on average). He et al. proposed $\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}$ specifically for layers with ReLU (or similar) activations [53]. This is often implemented as sampling $w \sim \mathcal{N}(0, \frac{2}{n_{in}})$ for Gaussian initialization [53], or an appropriate uniform range. Intuitively, with ReLU, only ~50% of neurons are active (not zero) for random inputs, so to keep the same variance going into the next layer, the weights need to be a bit larger (hence 2/fan_in instead of 1/fan_in).

These initialization schemes help start training in a regime where signals are neither too small (causing gradients to vanish) nor too large (causing saturation or blowing up). They *don't* guarantee things won't go haywire during training, but they give a fighting chance.

Variance through layers: Suppose you have a layer: $h = \phi(Wx)$, where W has entries with variance σ_w^2, and ϕ is an activation function. If x has variance σ_x^2, one can derive the

variance of h in terms of σ_w^2, σ_x^2 and properties of ϕ. For linear ϕ, $\mathrm{Var}(h) = n_{in} \sigma_w^2 \sigma_x^2$ (if inputs are i.i.d.). To maintain $\mathrm{Var}(h) = \sigma_x^2$ (the same as input), you'd set $n_{in}\sigma_w^2 = 1$, i.e. $\sigma_w^2 = 1/n_{in}$ – which is exactly LeCun initialization [52]. For ReLU, there's a factor of 1/2 (because ReLU outputs 0 half the time for symm input), leading to $\sigma_w^2 = 2/n_{in}$, which is He init. The Glorot formula $\frac{2}{n_{in}+n_{out}}$ is a compromise for cases like tanh where you also consider backward variance. The derivations of these can be found in the respective papers, but this is the gist.

Bias initialization: Biases are often just initialized to zero (or small constants). E.g., for ReLU networks, some recommend a small positive bias like 0.1 to avoid initial "dead" ReLUs [54], but it's not always necessary. Most frameworks default biases to 0.

In summary, **proper initialization** ensures at start: - Activations have similar scale across layers. - Gradients have similar scale across layers (so no layer is starving or dominating). This is one of the first lines of defense against training difficulties.

Normalization (BatchNorm, LayerNorm, etc.)

Normalization layers actively adjust the distribution of activations during training. The prime example is **Batch Normalization (BatchNorm)**. BatchNorm (Ioffe & Szegedy, 2015) was a breakthrough that allowed much faster and more stable training of very deep networks. It works by normalizing the *activations* of a layer for each minibatch. In a BatchNorm layer, for each feature k (e.g., each channel in a CNN or each component of a hidden layer output), you compute the batch's mean $\mu_B^{(k)}$ and variance $(\sigma_B^{(k)})^2$. Then you normalize:

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_B^{(k)}}{\sqrt{(\sigma_B^{(k)})^2 + \epsilon}},$$

where i indexes an example in the batch and ϵ is a small constant for numerical stability [55]. This produces normalized activations with mean 0 and variance 1 (approximately) for that batch [56]. Then BatchNorm applies a learned linear transformation:

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)},$$

where γ, β are learned parameters (scale and shift) per feature [57]. The γ,β allow the normalized output to still represent any distribution if needed (so the layer can learn to undo the normalization for certain neurons if that's optimal). Initially $\gamma=1,\beta=0$, so it starts as standard normalization.

Effect: BatchNorm makes the network **learn on a stable distribution of activations**. Each layer doesn't have to chase a moving target as much, a phenomenon BatchNorm's authors described as reducing *internal covariate shift*. Empirically, BatchNorm lets us use higher learning rates and be less careful about initialization, and it often improves convergence speed dramatically.

During training, BatchNorm uses batch statistics; at inference, it uses running averages of μ and σ^2 so that the normalization is deterministic.

Layer Normalization, etc.: BatchNorm's drawback is it depends on batch statistics – for very small batch sizes, it's noisy, and for recurrent networks or others where batch dims aren't i.i.d., it's not ideal [58]. Other normalization techniques address this: - **Layer Normalization:** computes mean/var across the features of a single example (instead of across the batch for each feature). Often used in Transformers (which prefer not to be batch-dependent). It normalizes each example's hidden units to mean 0, var 1. - **Instance Norm, Group Norm:** other variants dividing the dimensions differently. - **Weight Normalization:** normalizes the weights instead of activations.

The basic idea remains: **keep distributions well-behaved.** By maintaining roughly zero-mean, unit-variance activations, we ensure that layers operate in a similar regime throughout training.

Normalization and Gradient Flow: Normalization can also help gradients. For example, BatchNorm has a slight regularizing effect (adding noise from batch stats) and ensures that the gradient with respect to input is bounded (BatchNorm has derivative $\frac{\gamma}{\sqrt{\sigma^2+\epsilon}}$ applied to centered inputs, so it rescales gradients too). In fact, one can show BatchNorm bounds the gradient propagation to some extent [59] [60].

A quick example: Imagine a deep network without normalization: if one layer outputs very large values, the next layer's inputs are large, which could saturate something like a tanh, yielding near-zero gradients. BatchNorm would tame those large values back to N(0,1), avoiding saturation. Or if one layer's outputs had tiny variance, BatchNorm would boost them, avoiding signals that are too small to be useful.

In practice, **BatchNorm** is commonly used in CNNs and some fully-connected nets; **LayerNorm** is used in Transformers/RNNs; and more recently, **GroupNorm** or others if batch sizes are variable. These are implementation details, but the take-home message is: *Normalization helps maintain stable variance and gradient flow, complementing good initialization.* It's one of the reasons we can successfully train networks tens or hundreds of layers deep.

Calibration Basics (ECE)

After training a classifier, we often interpret its output probabilities as confidence levels. A model is **calibrated** if these confidence levels reflect true probabilities of correctness. For example, if a model says "90% probability of class A," it should be correct about 90% of the time when it says that. Calibration is important especially in applications where decision-making or uncertainty matters (e.g., medical diagnosis, weather forecasting as classic examples). A well-calibrated model allows downstream users to trust the probabilities.

Confidence Calibration: Formally, a classifier with prediction $\hat{y}$ and confidence $\hat{p} = \max_y q(y|x)$ is *calibrated* if: for all confidence levels c,

$$P(\text{correct} \mid \hat{p} = c) = c.$$

In practice, we approximate this by grouping predictions into bins of confidence (say [0.0–0.1), [0.1–0.2), ..., [0.9–1.0]) and compare the average confidence to the actual accuracy in each bin [61] [62]. A **reliability diagram** plots the expected accuracy vs confidence. A perfectly calibrated model would produce a diagonal line (confidence = accuracy in each bin) [61] [62]. Many deep networks are **miscalibrated** – often they are

over-confident (their predicted probabilities are too high relative to actual accuracy) as noted by Guo et al. (2017).

Expected Calibration Error (ECE): This is a popular summary metric for calibration. ECE takes the weighted average of the gaps between accuracy and confidence across these bins. If we denote: - B_i as the set of indices of predictions whose confidence falls into bin i, - p_i = average model confidence in bin i, - a_i = accuracy in bin i (fraction of predictions in B_i that were correct), - $|B_i|$ the number of points in bin i out of N total,

then **ECE** (with L1 norm) is:

$$\mathrm{ECE} = \sum_{i=1}^{M} \frac{|B_i|}{N} \left|a_i - p_i\right|.$$

This is essentially a weighted absolute difference between the model's accuracy and confidence, summed over bins [63] [64]. If a model is perfect calibrated, $a_i = p_i$ for all bins, so ECE = 0. Larger ECE means on average there is a gap between confidence and actual accuracy.

For example, suppose we have 100 predictions, and in the bin [0.8–0.9] there are 20 samples with average confidence 0.85, but only 70% (0.70 accuracy) of those are correct. Then for that bin, $|a_i - p_i| = |0.70 - 0.85| = 0.15$, and it contributes $\frac{20}{100}\cdot 0.15 = 0.03$ to ECE. We do this for all bins.

Sometimes they report **MCE (Maximum Calibration Error)** which is $\max_i |a_i - p_i|$ (worst-case bin gap) [65], or **RMSCE** (root-mean-squared error) [66]. But ECE (L1) is most common.

Using ECE: ECE is an *approximation* to the true calibration error (which would be an integral over confidences). It's simple and gives a single number. However, it has drawbacks: it depends on the binning strategy (number of bins M, spacing) [67], and it only looks at the top prediction confidence. It might mask issues in multi-class calibration (one can also compute class-wise ECE, etc.) [68]. Nonetheless, it's a good first check. For instance, if your model has 95% accuracy but is always 99% confident, it's over-confident and ECE will reflect that gap.

Calibration Techniques: If a model is not well-calibrated, one can post-process it with methods like **Platt Scaling** (learn a temperature scalar on logits on a validation set) or **Isotonic Regression**, or train with objectives that encourage calibration (e.g., add a regularizer, or use techniques like Mixup or label smoothing which often improve calibration). But those are beyond our scope here. Our focus is understanding what calibration means and how to measure it.

To summarize: - **Calibration** is about aligning confidence with correctness likelihood. - **ECE** is a scalar metric measuring the average discrepancy between confidence and accuracy [69] [64]. - Ideally, you want low ECE (e.g., <1% or <0.01) especially in critical applications. A perfectly calibrated model has ECE = 0 (rare in practice without explicit calibration). - Reliability diagrams are a great visual tool: the closer the plot is to diagonal, the better.

It's worth emphasizing that a model can be accurate but miscalibrated, or calibrated but not very accurate. We generally want high accuracy *and* calibration. If forced to choose (e.g., in competitions), accuracy often

wins out, but for deployed systems, calibration is key for making reliable decisions (like whether the model is uncertain and needs to defer to a human).

Basic Stats for Evaluation: Paired Bootstrap and Multiple Comparisons

When we evaluate models, especially when comparing two models or analyzing results across multiple tasks, statistical rigor is needed to ensure differences are *real* and not just due to chance. Here we cover two important concepts: **paired bootstrap testing** and the **multiple comparisons problem**.

Paired Bootstrap Significance Testing

Suppose we have two models A and B and we want to know if A is significantly better than B (e.g., in accuracy or BLEU score on some test set). A simple approach might be a t-test treating each test example's outcome as independent – but in ML evaluation, we often have limited test sets and weird, non-normal distributions for metrics. Instead, a common technique (especially in NLP) is the **paired bootstrap** (Koehn 2004 is a famous reference in MT evaluation).

How it works: The idea of a **bootstrap** is to resample your existing data to simulate variability. For paired bootstrap in model comparison, we: - Have a test set of N data points (e.g., sentences). Both model A and B produce outputs for each of these N items, and we compute a metric (accuracy, BLEU, etc.) for each model on the whole set. - To estimate the confidence in the difference, we create many *resampled test sets* by sampling **with replacement** N items from the original N (so some items repeat, some omitted). For each resampled set, compute model A's score and model B's score. - Count in how many of these resamples A's score > B's score (or vice versa). This gives a distribution of the score difference under the assumption the original test set is representative.

In a paired setup, we preserve the **pairing** of outputs: each resampled test set picks certain indices and takes both models' outputs for those indices [70] [71]. This way, differences due to particular examples are kept aligned between models.

If model A is truly better, it will outperform B on a large majority of these bootstrap replicates. If the difference is not significant, A might win on ~half and B on half. We can then derive a p-value (e.g., p = fraction of replicates where B >= A if we test "A > B").

Example: Let's say out of 1000 bootstrap samples, A had higher accuracy in 980 of them. That suggests A is very likely better (approx p = 0.02 if we test at 95% confidence). If A only wins 600/1000, that's p ~ 0.4 (not significant).

The **paired** aspect ensures that each resample maintains the dependency between model outputs on the same example – this often yields lower variance in estimates of difference, making the test more sensitive [71]. In contrast, an *unpaired* bootstrap would sample results for each model independently, losing that coupling information.

Why use bootstrap? Because we typically don't know the true distribution of performance over all possible inputs, but by resampling the test set, we approximate the uncertainty due to having a finite test set. It's a non-parametric method (doesn't assume normality, etc.).

Caveats: The bootstrap assumes the test set is representative of the population of interest [72]. If the test set is small or biased, resampling it might not capture all forms of variance. Also, bootstrap doesn't account for *variance due to training*, e.g., different random seeds (it treats the models as fixed). So if random initialization or training data order can lead to different model outcomes, ideally you test across multiple runs.

Mathias Müller's blog points out that **paired bootstrap doesn't account for variance across training runs** – so if model differences are due to luck from initialization, the bootstrap might still say "significant" because on this fixed pair of models A and B, A is better on basically every resample [73]. In other words, it captures test-set uncertainty but not training uncertainty. A rigorous study might involve multiple training runs for each model to incorporate that randomness as well.

Despite limitations, **paired bootstrap is widely used** for comparing models on metrics like BLEU, accuracy, etc., especially in competitions and research papers to back up claims "model A is better than B ($p<0.05$)". It's computationally easy and doesn't require additional data.

Multiple Comparisons

In experiments, it's common to compare *many* models, or evaluate a model on *multiple metrics or datasets*. The **multiple comparisons problem** (also known as multiple testing or *"look-elsewhere effect"*) is that the more comparisons you make, the more likely you are to find a "significant" difference just by chance [74] [75].

For example, if you test 20 independent hypotheses at significance level 0.05, on average 1 will be a false positive (5% of 20). In machine learning, one might run a model on 10 datasets and find it significantly better on 2 of them at $p<0.05$ – but some of those could be spurious findings because of multiple testing.

Controlling for multiple comparisons: There are statistical methods: - **Bonferroni correction:** Simplest, but very conservative. If you have m tests and you want overall $\alpha=0.05$, test each at α/m. E.g., 10 tests, require each to be $p < 0.005$ to call it significant. This guarantees the family-wise error rate ≤ 0.05 [76], but it can be overly strict if tests are not independent. - **Holm–Bonferroni, Benjamini-Hochberg (FDR control):** Less conservative, especially BH which controls expected false discovery rate rather than probability of any false discovery. These are common in e.g. genomics or any domain with lots of comparisons [77]. - **ANOVA / Friedman tests:** If comparing multiple models across multiple datasets, use a test designed for multiple comparisons (e.g., Friedman test with post-hoc Nemenyi, as recommended by Demšar (2006) for ML classifier comparisons) [78].

In practical ML terms: - If you evaluate a new model on 10 benchmarks and only 1 shows improvement with $p=0.01$, be cautious; that might happen by chance. Seeing consistent improvement across many benchmarks strengthens the evidence. - If you tune hyperparameters extensively and pick the best model, you've implicitly done multiple comparisons (each hyperparam setting is a "comparison"). It's important then to have a separate test set or use cross-validation to avoid **selection bias**. - When reporting results, if

you perform significance tests on multiple datasets, you should mention if you adjusted for multiple testing.

A humorous example of the multiple comparisons problem is finding correlations in random data: e.g., with enough random pairs of time series, you might "discover" that the number of people who drowned falling into a pool correlates with films Nicolas Cage appeared in (a classic spurious correlation example). Without correction, one might falsely declare significance [79] . Multiple comparisons techniques aim to prevent such false discoveries by effectively raising the bar for significance as the number of comparisons grows [76] .

In summary: Whenever many comparisons are made (multiple models, datasets, metrics, or repeated peeking at data), be aware that *some differences will appear significant by random chance.* Apply corrections or better experimental design: - Prefer evaluating a single hypothesis on a reserved test set (avoid testing 100 things on the same test set). - If multiple tests must be done, use appropriate corrections and report them.

As a researcher or practitioner, you should plan your statistical tests before looking at the data (to avoid "p-value hacking"). And remember that significance != importance; a tiny improvement can be statistically significant with large sample size but might not matter in practice, while a large improvement on a small test set might not reach $p<0.05$ yet is clearly meaningful. Use practical judgment in conjunction with statistical significance.

Exercises and Reflections

To solidify your understanding, consider the following exercises and questions. You can often use computational experiments or AI assistants to explore these (no manual coding required, but it can help). These will test concepts from each section:

1. **Derive Cross-Entropy from MLE:** Show analytically that for a classifier with one-hot labels, minimizing the negative log-likelihood is equivalent to minimizing the cross-entropy between the empirical distribution and the model distribution. *(Hint: Start from the definition $L = -\sum_i \log q_\theta(y^{(i)}|x^{(i)})$ and relate it to $H(p,q)$.)* What assumption about p (the true distribution of labels given x) makes this equivalence exact? [5] [6]

2. **Cross-Entropy and KL Practice:** Assume a true distribution $p = (0.4, 0.6)$ and two model distributions: $q_1 = (0.4, 0.6)$ and $q_2 = (0.7, 0.3)$. Compute $H(p,q_1)$ and $H(p,q_2)$ and verify that the difference equals the KL divergence $D_{KL}(p|q_2)$ (since q_1 is perfect, $H(p,q_1)=H(p)$). Confirm that $H(p,q_2) = H(p) + D_{KL}(p|q_2)$. This will give you numerical intuition for cross-entropy and KL [10] . *(You can do this calculation by hand or write a short script.)*

3. **Perplexity Calculation:** You have a language model that assigns a total log-likelihood of -50 (natural log) to a test set of 10 sentences, with total 100 words. What is the cross-entropy per word in nats and in bits? What is the perplexity per word? Now, suppose another model has a per-word perplexity of 120. Which model is better? *(Demonstrate the calculations: cross-entropy = 50/100 = 0.5 nats/word, convert to bits by dividing by ln2, etc.)* [16]

4. **Understanding Label Smoothing and KL:** Label smoothing replaces a one-hot target like $(0,1,0,0)$ with a softened version, e.g. $(0.05, 0.85, 0.05, 0.05)$ if smoothing factor is 0.1. Explain how this can be seen as adding a regularization term involving KL divergence. *(Hint: The loss becomes $0.85(-\log q(y_{\text{correct}})) + 0.05(-\log q(\text{other}))...$ which is the original cross-entropy plus an extra term that penalizes peaky distributions. That extra term is proportional to KL between q and uniform.)* Why might this improve model generalization?

5. **Backprop Through a Simple Network:** Consider a 2-layer neural network: input x, first layer weights $W^{[1]}$ (shape $H \times \text{dim}(x)$), ReLU activation, second layer weights $W^{[2]}$ (shape $1 \times H$ for a single output regression). Write out the forward equations: $h = \max(0, W^{[1]} x)$, $\hat{y} = W^{[2]} h$. Now derive the gradients of a mean squared error loss $L=\frac{1}{2}(\hat{y}-y)^2$ with respect to $W^{[2]}$ and $W^{[1]}$. *(This is a manual backprop derivation. Check: $\partial L/\partial W^{[2]} = (\hat{y}-y) h^\top$. For $W^{[1]}$, need to account for ReLU derivative: nonzero only where $h>0$.)* Verify your results by plugging in small numbers (or using an AI autodiff tool) to see if the gradients match a numerical approximation. This will reinforce the mechanics of backprop.

6. **Attention Mechanism Exploration:** Take a tiny example of attention: say Q is 1×2, K is 3×2, V is 3×2 (so 3 key-value pairs, 2-dimensional). Choose small concrete values for Q, K, V (like $Q=[1,0]$, $K = [[1,1],[0,-1],[2,1]]$, $V = [[1,0],[0,1],[1,1]]$). Manually compute the attention output step by step:

7. Dot products $q k^T$ (a 1x3 vector of scores),
8. Apply softmax (perhaps do it by computing exps and normalizing),

9. Weighted sum with V. Now implement the same in a few lines of Python (or ask an AI assistant) to verify your manual computation. Experiment by scaling q or K larger and see how the softmax outcome changes – confirm that scaling by $\sqrt{d_k}$ would counteract that. This gives you a tangible sense of how attention focuses on certain keys.

10. **Initial Weight Distribution Check:** Write a small program (or use an AI) to simulate a forward pass of a 10-layer network with no non-linearities: $y = W^{[10]} W^{[9]} \cdots W^{[1]} x$. For simplicity, let x be a vector of ones. Initialize weights in two ways: (A) small random $\mathcal{N}(0, 0.01)$ and (B) Xavier initialization (variance $=1/n_{in}$). Run a forward pass and observe the output norms or variances at each layer. Which initialization keeps the scale more consistent across layers? 49 Now add a nonlinearity like ReLU and repeat (ReLU will drop negative sums to 0). Observe how variance changes and how He initialization helps. This exercise connects theory to practice on why proper initialization matters.

11. **BatchNorm Empirical Test:** Using a deep learning framework or an AI coding assistant, create two identical deep networks (say 5 dense layers of 100 units with ReLU) – one with BatchNorm after each layer, and one without. Initialize them the same (use the same random seed). Train both on a moderate dataset (even random data or a simple function approximation). Monitor the training loss or accuracy. Typically, the one with BatchNorm will train faster and maybe reach a better result. Explain why based on the theory of internal covariate shift and gradient propagation. *(If you don't code, just reason about it: without BatchNorm, if one layer's outputs blow up, the next layer training is compromised. BatchNorm keeps things in check.)*

12. **Calibration Curve Plot:** Suppose you have a binary classifier that outputs scores, and after thresholding at 0.5 you get these results on 1000 examples:

13. 800 predictions with confidence > 0.9, of which 720 are correct.
14. 100 predictions with confidence around 0.6, of which 70 are correct.

15. 100 predictions with confidence around 0.4, of which 40 are correct (note: confidence <0.5 means predicted class = 0 but we can treat confidence = 1-score for the positive class for calibration). Calculate the accuracy and average confidence in each group. Plot a reliability diagram (just conceptual: plot points (avg_confidence, accuracy) for each group). Is the model calibrated in those ranges? Compute the ECE for these three bins (take bins as [0.0,0.5), [0.5,0.8), [0.8,1.0] for instance). [63] [64] What does the ECE tell you here? *(The model likely is a bit overconfident in the >0.9 bin: 0.9 vs 0.90 accuracy is fine; in 0.6 bin: confidence 0.6 vs accuracy 0.7 -> underconfident or just one instance; etc.)* This exercise simulates reading a calibration report.

16. **Significance Testing Thought Experiment:** Imagine you have two models A and B, and you evaluate them on 1000 samples. Model A got 850 correct, model B got 830 correct. So accuracy 85% vs 83%. A bootstrap test yields a p-value ~0.20 for A's superiority (not significant). Explain what that means in practical terms. Now imagine instead you had 10,000 samples with accuracies 85% vs 83%, and now p ~ 0.03 (significant). What changed, and why does sample size affect significance even though the accuracy gap (2%) is the same? Finally, consider if those 1000 samples were actually 10 different tasks of 100 each, and A wins on 9 of 10 tasks – how might you analyze significance in that multi-task scenario? (This touches on multiple comparisons and also paired analysis per task, perhaps using a sign test or Wilcoxon signed-rank across tasks instead of pooling all data.)

17. **Multiple Hypotheses Illustration:** Suppose you train 20 different models (with different random seeds or architectures) and evaluate all on a validation set, picking the best performer. Discuss why the best model's performance might be an overestimate of true performance. How could you use a hold-out test set or cross-validation to get an unbiased estimate? *(Relate this to multiple comparisons: you basically did 20 comparisons implicitly by choosing the best of 20. The more models you try, the higher chance one lucked out on the validation data.)* What is "validation leakage" in this context? Reflect on how you would fairly compare 5 models on 3 datasets without falling prey to false discoveries.

By working through these exercises, you will deepen your understanding of the mathematical foundations and how they manifest in practice. Remember, concepts like cross-entropy, KL divergence, or calibration are not just theory – they guide everyday decisions in model training and evaluation. And techniques like backpropagation, initialization, and normalization are the unsung heroes that make modern deep learning possible. Keep these foundations at your fingertips as you tackle advanced topics, and you'll find you have a strong, principled base to stand on. Good luck, and happy learning!

Sources: The content above references and builds upon key texts and resources in machine learning and information theory, including Goodfellow et al. (2016) *Deep Learning* [3] [14], Murphy (2012) *Machine Learning: A Probabilistic Perspective*, Cover & Thomas (2006) *Elements of Information Theory*, as well as research articles and educational blogs for specific topics like optimization [30], attention mechanisms [41], and model calibration [63] [64]. These references provide additional depth and are recommended for further reading.

1 2 4 5 6 7 8 10 12 Cross Entropy, KL Divergence, and Maximum Likelihood Estimation - Lei Mao's Log Book
https://leimao.github.io/blog/Cross-Entropy-KL-Divergence-MLE/

3 11 13 14 15 Loss and Loss Functions for Training Deep Learning Neural Networks - MachineLearningMastery.com
https://machinelearningmastery.com/loss-and-loss-functions-for-training-deep-learning-neural-networks/

9 19 20 21 22 23 24 25 26 Evaluation Metrics for Language Modeling
https://thegradient.pub/understanding-evaluation-metrics-for-language-models/

16 17 18 Perplexity - Wikipedia
https://en.wikipedia.org/wiki/Perplexity

27 [PDF] Lecture slides for Chapter 3 of Deep Learning www ...
https://www.deeplearningbook.org/slides/03_prob.pdf

28 cs.toronto.edu
https://www.cs.toronto.edu/~hinton/coursera/lecture9/lec9.pdf

29 Why do we regularize the variational autoencoder with a normal ...
https://ai.stackexchange.com/questions/18390/why-do-we-regularize-the-variational-autoencoder-with-a-normal-distribution

30 31 36 37 38 39 Calculus on Computational Graphs: Backpropagation -- colah's blog
https://colah.github.io/posts/2015-08-Backprop/

32 33 34 35 CS231n Deep Learning for Computer Vision
https://cs231n.github.io/optimization-2/

40 41 43 46 47 Scaled Dot-Product Attention
https://www.emergentmind.com/topics/scaled-dot-product-attention

42 11.3. Attention Scoring Functions - Dive into Deep Learning
https://d2l.ai/chapter_attention-mechanisms-and-transformers/attention-scoring-functions.html

44 Demystifying the Attention Formula | by Dagang Wei - Medium
https://medium.com/@weidagang/demystifying-the-attention-formula-8f5ad602546f

45 scaled_dot_product_attention in scirs2_linalg::attention - Docs.rs
https://docs.rs/scirs2-linalg/latest/scirs2_linalg/attention/fn.scaled_dot_product_attention.html

48 49 50 51 52 53 54 58 Weight initialization - Wikipedia
https://en.wikipedia.org/wiki/Weight_initialization

55 56 57 59 60 Batch normalization - Wikipedia
https://en.wikipedia.org/wiki/Batch_normalization

61 62 Understanding Model Calibration - A gentle introduction and visual exploration of calibration and the expected calibration error (ECE)
https://arxiv.org/html/2501.19047v2

63 64 65 66 69 Calibration Error — PyTorch-Metrics 1.8.1 documentation
https://lightning.ai/docs/torchmetrics/stable/classification/calibration_error.html

67 [PDF] Measuring Calibration in Deep Learning - CVF Open Access
https://openaccess.thecvf.com/content_CVPRW_2019/papers/Uncertainty%20and%20Robustness%20in%20Deep%20Visual%20Learning/Nixon_Measuring_Calibration_in_Deep_Learning_CVPRW_2019_paper.pdf

68 An Introduction to Calibration - Medium
https://medium.com/@conorwalsh206/an-introduction-to-calibration-e5a9f56173af

70 71 72 73 Statistical significance testing - Mathias Müller
https://bricksdont.github.io/posts/2020/12/statistical-significance-testing/

74 75 76 79 Multiple comparisons problem - Wikipedia
https://en.wikipedia.org/wiki/Multiple_comparisons_problem

77 Statistical tests for multiple comparisons - Changyao Chen
https://changyaochen.github.io/multiple-comparisons-jun-2020/

78 Statistical Tests in Machine Learning Modeling - Medium
https://medium.com/data-science-collective/statistical-tests-in-machine-learning-modeling-2629e72bc7f4